Doc. no.	P1374R0
Date:	2018-11-22
Project:	Programming Language C++
Audience:	Library Evolution Working Group
	Evolution Working Group
	Library Working Group
Reply to:	Alisdair Meredith <ameredith1@bloomberg.net>

Resolving LWG #2307 for C++20: Consistently Explicit Constructors

Table of Contents

Revision History

Revision 0

Original version of the paper for the 2018 post-San Diego mailing.

1 Introduction

Library issue LWG #2307 is a request to put the library into a consistent state with respect to multi-argument explicit constructors. While the resolution and wording were approved by the LWG prior to the most recent San Diego meeting, the LEWG would like to revisit the subject, and requested the paper be pulled until a clear policy for explicit constructors has been agreed.

2 Stating the problem

2.1 What is the Current Guideline?

The current guideline was never explicitly put into print, but the informal rule-of-thumb followed by LWG (prior to the formation of LEWG) during the C++11 timeframe was:

- Single argument constructors that are not copy or move constructors should be marked as `explicit` unless there is a good reason not to.
- All other constructors should *not* be marked `explicit` unless the is a good reason to do so.

An example of wanting an implicit conversion came up in LWG #925 where we changed the standard to allow implicit conversion of rvalues of unique ownership smart pointers to `shared_ptr` (but not from lvalues).

An example of adding `explicit` to multiple-argument constructors can be seen in `pair` and `tuple`, where we want to reflect the implicit conversion rules for the wrapped type. This models the rules of aggregate initialization followed by the language in [dcl.init.aggr] using copy initialization for each member supplied with an initializer.

Note that the currently guideline is not clear in the presence of variadic constructors, which may also act as single-argument converting constructors, or even default constructors, depending on how they are invoked.

2.2 How Was The Current Guideline Formed?

The basis of the current guideline came with direction from EWG while trying to resolve LWG #1153. The question of how to review the whole library, and which combinations of arguments to make explicit, was considered from a high level only, and the EWG designers of the unified initialization syntax pointed out that it was intentional to support the multiple-argument cases constructing from a brace-initializer list, and the library should think very carefully before disabling an intended feature (through use of `explicit`). Lacking any guiding principle to apply explicit without experience, LWG followed the EWG encouragement and did not consider the issue further, closing the issue NAD.

We now have most of a decade's experience with the new forms of initialization, and some additional guidance is emerging. It may well be time to revisit that earlier decision.

2.3 Why Do We Have A Problem?

In the original C++98 standard, the `explicit` keyword had no effect unless a constructor was called with a single argument. As a matter of convenience, several classes (notably containers) provided constructors with a number of default arguments. If these constructors could be called with a single argument, they would also be marked `explicit`.

When C++11 changed the semantics so that `explicit` had a new meaning for these multiple-argument cases, it was not immediately noticed. Since then, there has been concern that the design inherited from the core language change was never the intended library design. In particular, the issue of `explicit` default constructors was handled for C++14 in issue

LWG #2193 and in the paper P0935R0 from Tim Shen, adopted at the Jacksonville 2018 meeting,

2.4 What Have We Been Doing About The Problem?

Since the revised meaning of `explicit` was discovered, the LWG has been processing an ongoing sequence of defect reports. See References for a list. The last part of the puzzle passed LWG review at the Batavia 2018 extra-curricular LWG meeting, and was expected to be adopted at the San Diego 2018 meeting. It is believed, after our most careful review of the current wording, that this should be the last such issue unless a new library is adopted without consideration of the informal convention.

2.5 Why Have We Not Solved The Problem?

The final paper resolving the legacy wording was Tentatively Ready for polling in San Diego, but the LEWG questioned whether the informal consistency rule being applied was the correct choice. LEWG requested that the final paper be withdrawn while they consider anew what the desired rule for applying `explicit` to constructors should be. In particular, concerns were raised that making allowing multi-argument constructors constructors to be implicit (the default, and current state for most of the library) could easily lead to mistakes with accidental constructions.

The minutes to not suggest anyone familiar with the history of the last time this was discussed with EWG was present to explain the current policy, nor was consultation with EWG considered.

Polls indicate a desire for LEWG to formulate the policy for LWG to apply, and minimal support for applying the current paper. There was no indication of who would do the work to propose a new policy, nor direction polls on what it should be, although they may have apparent and obvious to those present in the room. No timetable for the new policy was suggested, nor guidance given for new papers coming through the process.

2.6 Further Insights

During informal discussion since the polls were taken (and I was not present in the room to know if this reflects the feel of the group, or just a few individuals I spoke to since) there seemed to be a feeling that passing sequences of brace-lists as arguments to functions was fragile and confusing, much like passing a long sequence of `true` and `false` flags can make it difficult to understand which flag is being set, often leading to use of enumerations in such APIs. Making such constructors `explicit` would at least require tagging with the type to be constructed, which is the only elided information today, and the root of much of the problem. There was much less concern about using brace lists for return values, as the return type is clearer. Perhaps another tweak of the initialization rules in core would better address the concerns being raised by LEWG? At least it seems that the two groups should be talking about expected design and usage of the language feature, before setting a new library-wide policy.

The meeting minutes show concerns that there is code breakage with both adding and removing `explicit` from constructors in this context, so any change would be equally concerning. This discounts existing experience with the Library already making such changes in the past (see References below). In general, removing `explicit` (other than the single-argument case) allows code that would not compile before to compile, but does not break or change existing code without hitting corners of SFINAE detection that we historically discount, for fear of never being able to change the library in even the most benign ways. There would be a compatibility concern adding `explicit` where it is absent today (not proposed by P1163R0), and this may become an issue if LEWG do not expect to have their new guidelines in place for C++20.

3 Propose Solution

The proposed solution is to adopt P1163R0 for C++20, and close LWG issue #2307 as resolved. Meanwhile, EWG and LEWG can agree a preferred policy for a future standard (potentially C++20). Once such a policy is agreed, it will be much simpler to review the library and apply it, as the document will be moving from one consistent state to another, and the pattern of edits should be regular.

There are several reasons why we should immediately adopt P1163R0, even if another approach from the evolutionary groups is pending.

3.1 The Wording is Complete and Reviewed

First, the wording is complete, and approved by the Library Working Group. We are not saving any of the limited LWG resources by asking them to complete this task, as they have already completed the work. The final paper was pulled from the planned motions page at the request of LEWG in San Diego.

3.2 Applying a New Consistently Will Be Simpler

Putting the working paper into a consistent state will make it easier to apply an update for a preferred consistency in the future. The author of the new paper can search and apply a (presumably) simple mechanical transformation rather than having to look out for, and account for, accidentally explicit constructors that are a consequence of the history of the specification, and not a deliberately designed intent where you might go looking for them.

3.3 P1163R0 is (Essentially) Not A Breaking Change

Any change to C++ libraries can be observed though SFINAE tricks (and the use of Concepts in the future) so every change is potentially breaking in that sense. Putting aside such concerns, the changes proposed in P1163R0 are entirely enabling, turning previously non-compiling code into code with a well-defined meaning.

3.4 There is Still Time for a More Complete Resolution

We anticipate at least four more meetings, including ballot comment resolution, before C++20 is published. If there is a strong desire to replace the current rule of thumb with a more precise convention to follow, we will not be harmed by landing the completed thought, even if it would allow programs to compile that might not under a new convention. If we are to change convention to adopt a more widespread use of `explicit` then it *is* going to be a breaking change, and there is time to agree and adopt that new convention prior to publishing any new standard. By the same argument, we could defer adopting P1163R0 until the final ballot resolution meeting. Such a delay mostly gives up on the convenience of applying that new convention over an inconsistent document, leading to more LWG work to get that final resolution right.

4 Other Directions

Several other directions were considered by the author, in the event that the proposed solution is not acceptable to the committee.

4.1 Status Quo

Status quo is that LWG has a priority 2 Open issue that it cannot make progress on without direction from LEWG. C++20 is published with this issue Open, and we get more comfortable releasing standards with high priority issues. This is my least preferred option.

Optionally, lower the issue priority to 3 or 4, as we would not expect to make progress until after C++20, or at least not until ballot resolution.

4.2 Revert All C++20 Changes

This is a more aggressive notion of status quo, resetting all use of `explicit` in constructors back to the C++17 state, pending eventual guidance on how to apply a consistent transformation to all library clauses of the

standard. At a minimum, this would mean reverting paper P0935R0, but also reviewing the issues list for issues in WP status that may have been impacted by this discussion.

This option may would compatible with also adopting one of the following choices.

4.3 Move Issue to LEWG for C++20

LWG moves the issue to LEWG status, expecting guidance on the new principle in time to apply it to C++20. This will mean working on the issue in preference to some other work, and finding LWG time to review and apply a new direction consistently across the library. This may be desirable, but seems a poor use of resources at this stage of the process, with so many other papers vying for attention.

4.4 Invite LWG to Propose Direction to LEWG for C++20

This is similar to option 4.2, only LWG are responsible for proposing a new direction that they can consistently apply in this time-frame, and LEWG get veto privilege over any such direction. This reflects where we are today, leaving the priority issue in LWG, but taking more time to find, propose, and word a new solution.

4.5 Move Issue to LEWG, With No Target

Move the issue to LEWG status, without direction for a time-frame. LEWG will prioritize this issue according to their interest level relative to the rest of their workload, and the standard remains in an inconsistent (but repeatedly shipped) state.

4.6 Move Issue to LEWGI

As we have no clear direction at the moment, we could move the work all the way out to LEWGI. Close the issue as NAD in on the Library Issues List, and invite interested parties to prepare presentations for LEWGI to incubate, until there is consensus on a new direction worth LEWG time to review.

4.7 Send Back to EWG to Ammend Initialization Rules for Simpler Consistency

Set the issue to the rarely used EWG status, and send back over to the language evolution group to tweak the initialization rules further, so that an intuitive direction for LEWG to apply to the standard library emerges.

5 Formal Wording

Apply the wording changes contained in P1163R0

6 Acknowledgements

Thanks to Nevin Lieber for performing the initial library review and providing wording in P1163R0.

7 References

- P1163R0 Explicitly Implicifying explicit Constructors, Nevin Lieber
- P0935R0 Eradicating unnecessarily explicit default constructors from the standard library, Tim Song
- LWG #2307 Should the Standard Library use `explicit` only when necessary?
- LWG #2193 Default constructors for standard library containers are explicit
- LWG #2051 Explicit `tuple` constructors for more than one parameter

- LWG #1430 `unordered_multiset` constructor accepting an allocator as a single parameter should be explicit
- LWG #1429 `unordered_set` constructor accepting an allocator as a single parameter should be explicit
- LWG #1428 `unordered_multimap` constructor accepting an allocator as a single parameter should be explicit
- LWG #1427 `unordered_map` constructor accepting an allocator as a single parameter should be explicit
- LWG #1426 `multiset` constructor accepting an allocator as a single parameter should be explicit
- LWG #1425 `set` constructor accepting an allocator as a single parameter should be explicit
- LWG #1424 `multimap` constructor accepting an allocator as a single parameter should be explicit
- LWG #1423 `map` constructor accepting an allocator as a single parameter should be explicit
- LWG #1400 FCD `function` does not need an explicit default constructor
- LWG #1399 `std::function` does not need an explicit default constructor
- LWG #925 `shared_ptr`'s explicit conversion from `unique_ptr`
- LWG #789 `xor_combine_engine(result_type)` should be explicit
- LWG #1153 Standard library needs review for constructors to be explicit to avoid treatment as initializer-list constructor